دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۵

طراحی ضرب‌کننده‌ی بوث

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

سال تحصیلی ۱۴۰۴–۱۴۰۵

۱.۵ مقدمه و اهداف

هدف این آزمایش، طراحی یک واحد ضرب‌کننده است که عمل ضرب دو عدد علامت‌دار را با الگوریتم بوث[۱] انجام می‌دهد. مطابق دستورکار، مسیر داده[۲] و واحد کنترل[۳] باید جداگانه طراحی و سپس به یکدیگر متّصل شوند. افزون بر این، دستورکار تصریح می‌کند که شیفت‌دهندهٔ مضروب‌فیه[۴] باید توانایی جابه‌جایی بیش از یک بیت در هر پالس ساعت را داشته باشد تا طرح نسبت به ضرب متعارف شیفت و جمع[۵] تسریع داشته باشد.

طرح پیاده‌سازی‌شده یک ضرب‌کنندهٔ چهاربیتی علامت‌دار در نمایش متمّم دو[۶] است؛ هر دو عملوند در بازهٔ -8 تا $+7$ قرار دارند و حاصل‌ضرب هشت‌بیتی و علامت‌دار در بازهٔ -56 تا $+64$ است. توصیف مدار در زبان Verilog و به‌صورت رفتاری انجام شده و در سه پیمانهٔ مجزّا سازمان یافته است: مسیر داده، واحد کنترل و پیمانهٔ بالا که این دو را به هم می‌دوزد.

روند انجام آزمایش

ابتدا الگوریتم بوث و صورت تعمیم‌یافتهٔ آن با شیفت چندبیتی استخراج و بر پایهٔ آن، تقسیم وظایف میان مسیر داده و واحد کنترل تعیین شد. سپس سه پروندهٔ توصیف سخت‌افزار نوشته و در ModelSim همگردانی شدند. برای ارزیابی، یک میز آزمون[۷] خودآزما[۸] ساخته شد که هر ۲۵۶ زوج ورودی ممکن را می‌آزماید و خروجی مدار را با عملگر ضرب مرجع زبان مقایسه می‌کند. پس از تأیید درستی در شبیه‌سازی و بررسی دقیق شکل موج‌ها، همان سه پروندهٔ سنتزپذیر در Quartus II همگردانی و سنتز شدند و نمای سطح انتقال ثبات‌ها[۹]، گزارش مصرف منابع و تحلیل زمانی طرح بررسی شد.

[1] Booth's Multiplication Algorithm
[2] Datapath
[3] Control Unit
[4] Multiplier
[5] Shift & Add Multiplication
[6] Two's Complement
[7] Testbench
[8] Self-Checking Testbench
[9] Register Transfer Level (RTL) View

۲.۵ الگوریتم بوث و شیفت چندبیتی

الف) صورت پایه‌ی الگوریتم

الگوریتم بوث برای ضرب دو عدد در نمایش متمّم دو به کار می‌رود و مزیّت آن، پرهیز از رسیدگی جداگانه به علامت عملوندهاست. سه ثبات اصلی در این الگوریتم عبارت‌اند از انباره[۱۰] A که ابتدا صفر است، ثبات Q که مضروب‌فیه را نگه می‌دارد و ثبات M که مضروب[۱۱] را نگه می‌دارد. افزون بر این‌ها، یک تک‌بیت کمکی به نام Q_{-1} در سمت راست Q قرار می‌گیرد و مقدار اوّلیّهٔ آن صفر است.

در هر تکرار، زوج بیت (Q_0, Q_{-1}) بررسی و بر اساس آن یکی از سه عمل زیر انجام می‌شود:

$$(Q_0, Q_{-1}) = \begin{cases} (0,1) & A \leftarrow A + M, \\ (1,0) & A \leftarrow A - M, \\ (0,0) \text{ یا } (1,1) & \text{بدون تغییر.} \end{cases}$$

سپس کلّ ثبات پیوستهٔ $\{A, Q, Q_{-1}\}$ یک بیت به راست شیفت حسابی[۱۲] داده می‌شود؛ یعنی بیت علامت A در پرارزش‌ترین جایگاه تکرار می‌گردد. برای عملوند n بیتی، این تکرار n بار انجام می‌شود و در پایان، حاصل‌ضرب $2n$ بیتی در $\{A, Q\}$ قرار دارد.

مبنای درستی این روش آن است که یک دنبالهٔ پیوسته از یک‌ها در مضروب‌فیه، از بیت i تا بیت j، برابر است با

$$\sum_{k=i}^{j} 2^k = 2^{j+1} - 2^i.$$

بنابراین به‌جای $j - i + 1$ بار جمع، تنها یک تفریق در ابتدای دنباله و یک جمع در انتهای آن کافی است. زوج $(1,0)$ دقیقاً آغاز چنین دنباله‌ای و زوج $(0,1)$ پایان آن را نشانه‌گذاری می‌کند. این ویژگی، کدگذاری مجدّد بوث[۱۳] نامیده می‌شود.

ب) شیفت چندبیتی و تسریع مدار

در صورت پایه، حتّی وقتی زوج بیت‌ها $(0,0)$ یا $(1,1)$ است و هیچ عمل حسابی لازم نیست، باز هم یک پالس ساعت صرف شیفت یک‌بیتی می‌شود. ایدهٔ تسریع خواسته‌شده در دستورکار این است که چند بیت

[10] Accumulator
[11] Multiplicand
[12] Arithmetic Right Shift
[13] Booth Recoding

همسان مجاور در Q هم‌زمان و در یک پالس ساعت رد شوند. برای این کار، به یک شیفت‌دهندهٔ بشکه‌ای[۱۴] نیاز است که اندازهٔ شیفت آن در زمان اجرا تعیین می‌شود.

برای یافتن تعداد بیت‌هایی که می‌توان در یک گام مصرف کرد، مکان نخستین تغییر میان بیت‌های مجاور Q از سمت کم‌ارزش لازم است. اگر $D = Q \oplus (Q \gg 1)$ تعریف شود، بیت iام D تنها هنگامی یک است که $Q_i \neq Q_{i+1}$ باشد. در پیاده‌سازی، پرارزش‌ترین بیت D به‌اجبار یک می‌شود:

$$\text{differences} = \big(Q \oplus (Q \gg 1)\big) \ \vee \ 1000_2.$$

این نقاب‌گذاری تضمین می‌کند که حتّی در حالت‌های $Q = 0000$ و $Q = 1111$ نیز دست‌کم یک بیت روشن وجود دارد و مقدار خروجی همواره در بازهٔ 1 تا 4 می‌ماند. جایگاه کم‌ارزش‌ترین بیت روشن differences، که در طرح با نام rightMost شناخته می‌شود، برابر طول دنبالهٔ بیت‌های همسان انتهای Q است. اندازهٔ شیفت این گام از رابطهٔ زیر به دست می‌آید:

$$\text{booth_bits} = \min\big(\text{bits_remaining},\ \text{rightMost}\big),$$

که در آن bits_remaining تعداد بیت‌های پردازش‌نشدهٔ باقی‌مانده است. حضور این کمینه‌گیری ضروری است، زیرا در غیر این صورت شیفت پایانی می‌توانست از مرز چهاربیتی عبور کند و بیت‌های مضروب‌فیه را بیش از اندازه جابه‌جا سازد.

درستی این تعمیم از آنجا ناشی می‌شود که اگر k بیت انتهایی Q با هم برابر باشند، زوج‌های بررسی‌شده در $k-1$ تکرار بعدی همگی از نوع $(0,0)$ یا $(1,1)$ و در نتیجه بدون عمل حسابی خواهند بود. پس می‌توان عمل حسابی گام جاری را انجام داد و بلافاصله k بیت شیفت داد.

بدین‌ترتیب تعداد تکرارهای لازم برای عملوند چهاربیتی، به‌جای مقدار ثابت چهار، بین یک تا چهار تکرار متغیّر است. برای نمونه، $Q = 1111$ یا $Q = 0000$ تنها یک تکرار لازم دارد، حال آنکه $Q = 0101$ که بیت‌های آن متناوب‌اند، چهار تکرار می‌خواهد. این همان تسریعی است که دستورکار خواسته است.

ج) بیت نگهبان و پهنای ثبات‌ها

نکتهٔ ظریف طراحی، پهنای انباره است. در گام تفریق باید $A - M$ محاسبه شود و برای کوچک‌ترین مقدار چهاربیتی علامت‌دار، یعنی $M = -8 = 1000_2$، این عمل به $A + 8$ می‌انجامد؛ حال آنکه $+8$ در چهار بیت علامت‌دار نمایش‌پذیر نیست و سرریز[۱۵] رخ می‌دهد. به همین دلیل، انباره و ثبات مضروب هر دو پنج‌بیتی گرفته شده‌اند و مضروب هنگام بارگذاری با بسط علامت[۱۶] به پنج بیت تبدیل می‌شود. این بیت اضافی،

[14] Barrel Shifter
[15] Overflow
[16] Sign Extension

بیت نگهبان[۱۷] نامیده می‌شود و تنها در واسط نهایی حاصل‌ضرب هشت‌بیتی کنار گذاشته می‌شود.

۳.۵ معماری کلّی و پیمانه‌ی بالا

پیمانهٔ بالا با نام BoothMultiplier فقط نقش دوخت را بر عهده دارد: نمونه‌ای از مسیر داده و نمونه‌ای از واحد کنترل می‌سازد و سیگنال‌های فرمان و بازخورد را میان آن‌ها می‌بندد. سه فرمان load، arth و shiftRight از واحد کنترل به مسیر داده می‌روند و اندازهٔ شیفت با گذرگاه booth_bits منتقل می‌شود. در جهت مخالف، مقدار جاری ثبات Q از مسیر داده به واحد کنترل بازخورد می‌شود تا واحد کنترل بتواند در هر گام، طول دنبالهٔ بیت‌های همسان را از روی مقدار به‌روزشدهٔ Q و نه از روی ورودی اوّلیّه محاسبه کند. این بازخورد، هستهٔ سازوکار شیفت متغیّر است.

قرارداد شروع کار چنین است: سیگنال str دست‌کم برای یک لبهٔ بالارونده‌ٔ ساعت فعّال و سپس غیرفعّال می‌شود؛ آماده شدن نتیجه با یک شدن valid اعلام می‌گردد. خروجی هشت‌بیتی result از الحاق چهار بیت کم‌ارزش انباره و ثبات Q ساخته می‌شود.

```
module BoothMultiplier(
    input  wire [3:0] multiplicand,
    input  wire [3:0] multiplier,
    input  wire       clk,
    input  wire       str,
    output wire [7:0] result,
    output wire       valid
);

    wire       shiftRight;
    wire       load;
    wire       arth;
    wire [2:0] booth_bits;
    wire [3:0] acc_low;
    wire [3:0] q_value;

    BoothMultiplierDatapath datapath (
        .multiplicand   (multiplicand),
        .multiplier     (multiplier),
        .shiftRight     (shiftRight),
        .load           (load),
        .arth           (arth),
        .booth_bits     (booth_bits),
        .clk            (clk),
        .result         (acc_low),
        .new_multiplier (q_value)
    );

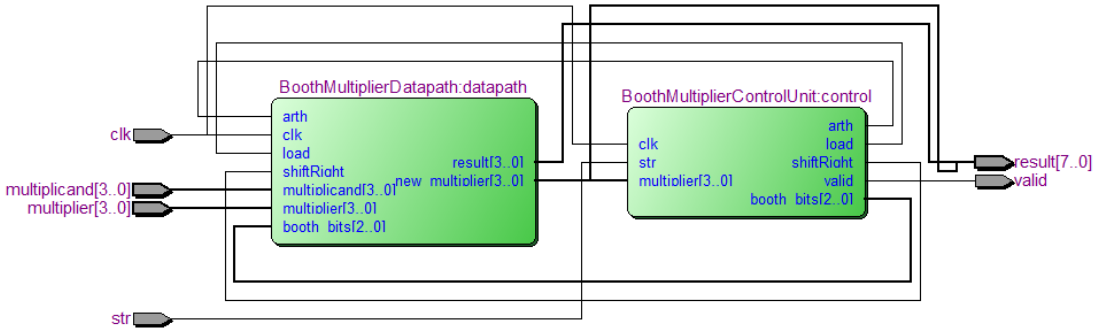
    BoothMultiplierControlUnit control (
        .multiplier (q_value),
        .clk        (clk),
        .str        (str),
        .valid      (valid),
        .load       (load),
        .arth       (arth),
        .shiftRight (shiftRight),
        .booth_bits (booth_bits)
```

[17] Guard Bit

```
38      );
39
40      // The internal accumulator has one guard/sign bit.  The guard bit is
41      // discarded only at the final 8-bit product interface.
42      assign result = {acc_low, q_value};
43
44  endmodule
```

کد ۱: پیمانهٔ بالا و اتّصال مسیر داده به واحد کنترل

نمای سطح انتقال ثبات‌های پیمانهٔ بالا نیز همین ساختار دو بلوکی را نشان می‌دهد: دو بلوک مجزّای BoothMultiplierDatapath و BoothMultiplierControlUnit با مسیرهای فرمان در یک جهت و مسیر بازخورد multiplier در جهت دیگر. جدایی مسیر داده و واحد کنترل که دستورکار خواسته بود، در این نما به‌روشنی دیده می‌شود.



شکل ۱: نمای سطح انتقال ثبات‌های پیمانهٔ بالا و جدایی مسیر داده از واحد کنترل

۴.۵ طراحی مسیر داده

مسیر داده چهار عنصر حافظه دارد: انباره accumulator با پنج بیت، ثبات مضروب temp_multiplicand با پنج بیت، ثبات q_reg با چهار بیت و تک‌بیت q_minus_one. برای آنکه شیفت حسابی چندبیتی بتواند بیت علامت را به‌درستی تکرار کند، هر سه ثبات نخست در یک سیگنال پیوستهٔ ده‌بیتی علامت‌دار به نام booth_register کنار هم قرار می‌گیرند و عملگر شیفت حسابی به راست ⋙ روی همین مقدار علامت‌دار اعمال می‌شود.

رفتار مسیر داده در هر لبهٔ بالارونده‌ٔ ساعت، بر پایهٔ سه فرمان ورودی و با اولویّت زیر تعیین می‌شود:

- load: انباره صفر، مضروب با بسط علامت در temp_multiplicand، مضروب‌فیه در q_reg و بیت q_minus_one صفر می‌شود.

- arth: بر پایهٔ زوج (Q_0, Q_{-1}) یکی از سه عمل جمع، تفریق یا حفظ مقدار روی انباره انجام می‌شود و سایر ثبات‌ها دست‌نخورده می‌مانند.
- shiftRight: کلّ booth_register به اندازهٔ booth_bits به راست شیفت حسابی داده می‌شود و مقدار حاصل دوباره میان سه ثبات توزیع می‌گردد.

```
module BoothMultiplierDatapath(
    input  wire [3:0] multiplicand,
    input  wire [3:0] multiplier,
    input  wire       shiftRight,
    input  wire       load,
    input  wire       arth,
    input  wire [2:0] booth_bits,
    input  wire       clk,
    output wire [3:0] result,
    output wire [3:0] new_multiplier
);

    // A and M use one extra sign/guard bit.
    reg signed [4:0] accumulator;
    reg signed [4:0] temp_multiplicand;
    reg        [3:0] q_reg;
    reg              q_minus_one;

    // Treat the complete Booth register as one signed quantity so that >>>
    // replicates the sign bit of A during a variable-length arithmetic shift.
    wire signed [9:0] booth_register;
    assign booth_register = $signed({accumulator, q_reg, q_minus_one});

    // Preserve the original module interface: high product nibble + Q nibble.
    assign result         = accumulator[3:0];
    assign new_multiplier = q_reg;

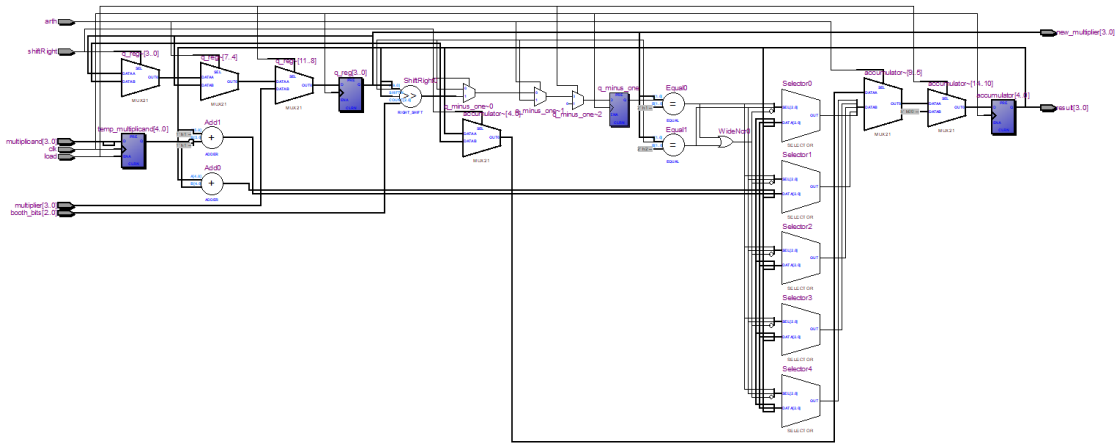
    always @(posedge clk) begin
        if (load) begin
            accumulator       <= 5'sd0;
            temp_multiplicand <= {multiplicand[3], multiplicand};
            q_reg             <= multiplier;
            q_minus_one       <= 1'b0;
        end
        else if (arth) begin
            case ({q_reg[0], q_minus_one})
                2'b01: accumulator <= accumulator + temp_multiplicand;
                2'b10: accumulator <= accumulator - temp_multiplicand;
                default: accumulator <= accumulator;
            endcase
        end
        else if (shiftRight) begin
            {accumulator, q_reg, q_minus_one} <= booth_register >>> booth_bits;
        end
    end

endmodule
```

کد ۲: مسیر داده‌ی ضرب‌کننده‌ی بوث با شیفت حسابی چندبیتی

در نمای سطح انتقال ثبات‌های مسیر داده، عناصر مورد انتظار دیده می‌شوند: ثبات‌های q_reg، temp_multiplicand، accumulator و q_minus_one، دو جمع‌کننده که یکی برای جمع و دیگری برای تفریق M به کار می‌رود، تسهیم‌کننده‌های انتخاب میان سه رفتار مرحلهٔ حسابی، و یک بلوک شیفت‌دهنده با برچسب RIGHT_SHIFT که ورودی اندازهٔ شیفت آن از گذرگاه booth_bits تغذیه می‌شود. وجود

همین بلوک با ورودی متغیّر، گواه آن است که سنتز به‌درستی یک شیفت‌دهندهٔ بشکه‌ای ساخته و نه زنجیره‌ای از شیفت‌های ثابت یک‌بیتی.



شکل ۲: نمای سطح انتقال ثبات‌های مسیر داده

۵.۵ طراحی واحد کنترل

واحد کنترل یک ماشین حالت متناهی[۱۸] با چهار حالت است:

- STATE_IDLE با کد 000: حالت آرام پیش از نخستین پالس شروع.
- STATE_ARITH با کد 001: مرحلهٔ حسابی؛ در این حالت arth فعّال است.
- STATE_SHIFT با کد 010: مرحلهٔ شیفت؛ در این حالت shiftRight فعّال است.
- STATE_DONE با کد 011: پایان کار؛ در این حالت valid فعّال است و ماشین در همین حالت می‌ماند تا پالس شروع بعدی برسد.

نکتهٔ طراحی مهم آن است که خودِ str به‌عنوان اجازهٔ بارگذاری هم‌زمان به کار رفته است؛ یعنی load = str. با این تصمیم، طرح به مقدار اوّلیّهٔ ثبات حالت وابسته نیست و یک لبهٔ شروع، هم مسیر داده و هم واحد کنترل را به‌طور کامل مقداردهی می‌کند. به همین دلیل، هر سه فرمان دیگر با شرط ¬str مقیّد شده‌اند تا در پالس شروع، بارگذاری با مرحلهٔ حسابی یا شیفت تداخل نکند.

شمارندهٔ bits_remaining در پالس شروع با مقدار چهار بارگذاری می‌شود و پس از هر شیفت به اندازهٔ booth_bits کاهش می‌یابد. هرگاه bits_remaining ≤ booth_bits شود، یعنی همین شیفت آخرین بیت‌های باقی‌مانده را مصرف می‌کند، ماشین به‌جای بازگشت به مرحلهٔ حسابی به حالت پایان می‌رود.

[18] Finite State Machine

محاسبهٔ rightMost در یک بلوک ترکیبی و با حلقهٔ for انجام می‌شود؛ این حلقه در زمان سنتز باز می‌شود و به یک زنجیرهٔ اولویّت ترکیبی تبدیل می‌گردد، نه به یک ساختار ترتیبی. دقّت شود که ورودی multiplier این پیمانه به مقدار جاری q_reg مسیر داده متّصل است و نه به ورودی بیرونی مدار؛ ازاین‌رو در هر گام، طول دنبالهٔ بیت‌های همسان روی مقدار شیفت‌خوردهٔ جاری سنجیده می‌شود.

```
module BoothMultiplierControlUnit(
    input  wire [3:0] multiplier,
    input  wire       clk,
    input  wire       str,
    output wire       valid,
    output wire       load,
    output wire       arth,
    output wire       shiftRight,
    output wire [2:0] booth_bits
);

    localparam [2:0] STATE_IDLE  = 3'd0;
    localparam [2:0] STATE_ARITH = 3'd1;
    localparam [2:0] STATE_SHIFT = 3'd2;
    localparam [2:0] STATE_DONE  = 3'd3;

    reg [2:0] state;
    reg [2:0] bits_remaining;

    // str itself is the synchronous load enable.  This avoids dependence on an
    // uninitialized state register at power-up: one asserted start edge fully
    // initializes both the control path and the datapath.
    assign load       = str;
    assign arth       = (!str) && (state == STATE_ARITH);
    assign shiftRight = (!str) && (state == STATE_SHIFT);
    assign valid      = (!str) && (state == STATE_DONE);

    // Find the first transition between neighboring bits of Q, scanning from
    // the LSB upward.  The forced MSB marker guarantees a value in 1..4.
    // Examples:
    //   4'b0011 -> 2  (two adjacent 1s can be consumed)
    //   4'b0000 -> 4  (four adjacent 0s can be consumed)
    //   4'b1111 -> 4  (four adjacent 1s can be consumed)
    wire [3:0] differences;
    reg  [2:0] rightMost;
    integer i;

    assign differences = (multiplier ^ (multiplier >> 1)) | 4'b1000;

    always @(*) begin
        rightMost = 3'd0;
        for (i = 1; i <= 4; i = i + 1) begin
            if (differences[i-1] && (rightMost == 3'd0))
                rightMost = i;
        end
    end

    assign booth_bits = (bits_remaining < rightMost) ? bits_remaining : rightMost;

    always @(posedge clk) begin
        if (str) begin
            state          <= STATE_ARITH;
            bits_remaining <= 3'd4;
        end
        else begin
            case (state)
                STATE_ARITH: begin
                    state <= STATE_SHIFT;
                end

                STATE_SHIFT: begin
                    if (bits_remaining <= booth_bits) begin
```

```
                        bits_remaining <= 3'd0;
                        state          <= STATE_DONE;
                    end
                    else begin
                        bits_remaining <= bits_remaining - booth_bits;
                        state          <= STATE_ARITH;
                    end
                end

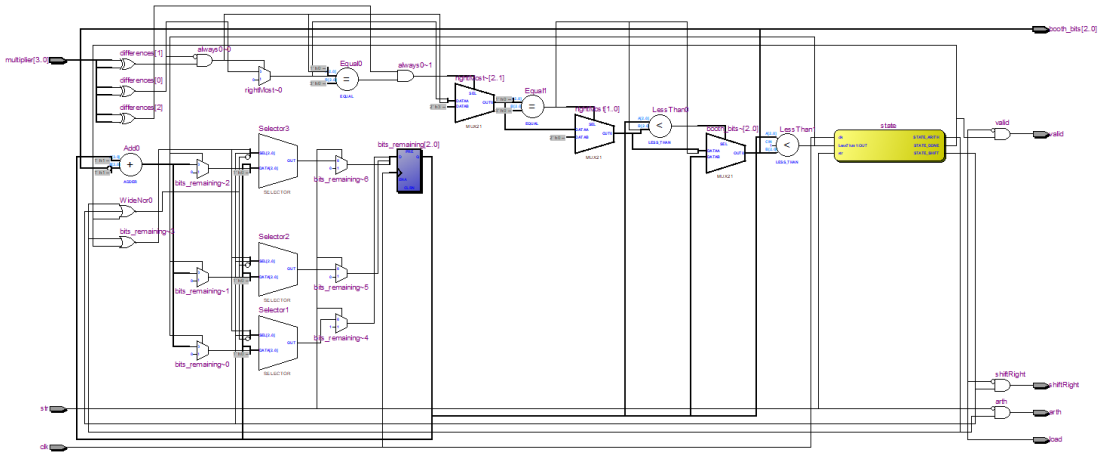
                STATE_DONE: begin
                    state <= STATE_DONE;
                end

                default: begin
                    // Safe idle behavior before the first start pulse.
                    state          <= STATE_IDLE;
                    bits_remaining <= 3'd0;
                end
            endcase
        end
    end

endmodule
```

کد ۳: واحد کنترل، ماشین حالت و محاسبه‌ی اندازه‌ی شیفت

نمای سطح انتقال ثبات‌های واحد کنترل، ثبات حالت state، شمارندهٔ bits_remaining، منطق دروازه‌ای تولید differences، زنجیرهٔ اولویّت محاسبهٔ rightMost و دو مقایسه‌کنندهٔ LessThan را نشان می‌دهد که یکی برای کمینه‌گیری booth_bits و دیگری برای شرط پایان به کار می‌روند.



شکل ۳: نمای سطح انتقال ثبات‌های واحد کنترل

۶.۵ میز آزمون و اجرا در ModelSim

میز آزمون به‌صورت خودآزما نوشته شده است و آزمون فراگیر[۱۹] انجام می‌دهد؛ یعنی هر ۱۶ مقدار ممکن مضروب را در هر ۱۶ مقدار ممکن مضروب‌فیه ضرب می‌کند و در مجموع ۲۵۶ حالت را می‌آزماید. با این پوشش کامل، حالت‌های مرزی مانند صفر، -1، $+7$ و به‌ویژه -8 که بحرانی‌ترین حالت طرح است، به‌طور خودکار پوشش داده می‌شوند.

ساعت با نیم‌دورهٔ ۵ نانوثانیه، یعنی دورهٔ ۱۰ نانوثانیه معادل ۱۰۰ مگاهرتز، تولید می‌شود. برای هر زوج ورودی، عملوندها در لبهٔ پایین‌روندهٔ ساعت اعمال و یک پالس شروع تمیز به طول یک دورهٔ ساعت ساخته می‌شود؛ اعمال ورودی‌ها در لبهٔ پایین‌رونده تضمین می‌کند که شرط‌های برپایی و نگهداری نسبت به لبهٔ فعّال نقض نشوند. سپس میز آزمون منتظر یک شدن valid می‌ماند. این انتظار با شمارندهٔ محدودیّت زمانی همراه است تا اگر واحد کنترل به هر دلیل در حلقه بیفتد، آزمون به‌جای معلّق ماندن، خطای TIMEOUT گزارش کند. در پایان، خروجی مدار با عملگر ضرب مرجع زبان مقایسه و نتیجهٔ هر حالت چاپ می‌شود.

```
`timescale 1ns/1ps

// ------------------------------------------------------------------------
// Self-checking exhaustive testbench.
// Tests every signed 4-bit pair: 16 x 16 = 256 products.
// Includes zero, -1, +7 and the critical -8 boundary automatically.
// ------------------------------------------------------------------------
module BoothMultiplierTestbench;

    reg        clk;
    reg        str;
    reg  [3:0] multiplicand;
    reg  [3:0] multiplier;
    wire [7:0] result;
    wire       valid;

    integer a;
    integer b;
    integer expected;
    integer errors;
    integer tests;
    integer cycles;

    BoothMultiplier dut (
        .multiplicand (multiplicand),
        .multiplier   (multiplier),
        .clk          (clk),
        .str          (str),
        .result       (result),
        .valid        (valid)
    );

    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial begin
        str          = 1'b0;
        multiplicand = 4'd0;
        multiplier   = 4'd0;
        errors        = 0;
        tests         = 0;
```

[19] Exhaustive Test

```
        // Exhaustive signed test: -8 .. +7 for both operands.
        for (a = -8; a <= 7; a = a + 1) begin
            for (b = -8; b <= 7; b = b + 1) begin

                // Apply operands and issue a clean one-cycle start pulse.
                @(negedge clk);
                multiplicand = a[3:0];
                multiplier   = b[3:0];
                str          = 1'b1;

                @(negedge clk);
                str = 1'b0;

                // Wait for completion, with a timeout to catch controller bugs.
                cycles = 0;
                while ((valid !== 1'b1) && (cycles < 12)) begin
                    @(negedge clk);
                    cycles = cycles + 1;
                end

                tests    = tests + 1;
                expected = a * b;

                if (valid !== 1'b1) begin
                    errors = errors + 1;
                    $display("TIMEOUT: %0d * %0d", a, b);
                end
                else if ($signed(result) !== expected) begin
                    errors = errors + 1;
                    $display("FAIL: %0d * %0d => got %0d (0x%02h), expected %0d",
                             a, b, $signed(result), result, expected);
                end
                else begin
                    $display("PASS: %0d * %0d = %0d", a, b, $signed(result));
                end
            end
        end

        $display("-------------------------------------------------");
        $display("Tests  : %0d", tests);
        $display("Errors : %0d", errors);
        if (errors == 0)
            $display("ALL 256 SIGNED 4-BIT TESTS PASSED.");
        else
            $display("TEST FAILED.");
        $display("-------------------------------------------------");

        $stop;
    end

endmodule
```

کد ۴: میز آزمون خودآزما با پوشش فراگیر ۲۵۶ حالت

پس از همگردانی سه پیمانهٔ طرح و میز آزمون در کتابخانهٔ work، فرمان‌های زیر در پنجرهٔ رونوشت ModelSim اجرا شدند. گزینهٔ -voptargs=+acc از بهینه‌سازی سیگنال‌های داخلی جلوگیری می‌کند و بدون آن، ثبات‌های درونی مسیر داده و واحد کنترل در پنجرهٔ موج در دسترس نمی‌بودند.

```
vsim -voptargs=+acc work.BoothMultiplierTestbench

view wave

add wave sim:/BoothMultiplierTestbench/clk
add wave sim:/BoothMultiplierTestbench/str
```

```
add wave sim:/BoothMultiplierTestbench/multiplicand
add wave sim:/BoothMultiplierTestbench/multiplier
add wave sim:/BoothMultiplierTestbench/result
add wave sim:/BoothMultiplierTestbench/valid

add wave sim:/BoothMultiplierTestbench/dut/datapath/accumulator
add wave sim:/BoothMultiplierTestbench/dut/datapath/temp_multiplicand
add wave sim:/BoothMultiplierTestbench/dut/datapath/q_reg
add wave sim:/BoothMultiplierTestbench/dut/datapath/q_minus_one
add wave sim:/BoothMultiplierTestbench/dut/datapath/booth_register

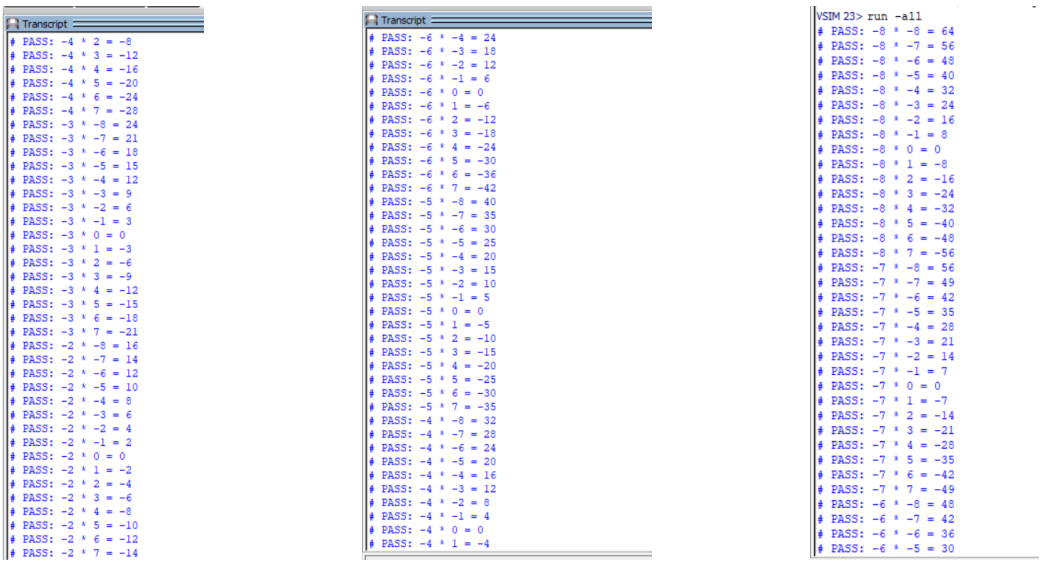
add wave sim:/BoothMultiplierTestbench/dut/control/state
add wave sim:/BoothMultiplierTestbench/dut/control/bits_remaining
add wave sim:/BoothMultiplierTestbench/dut/control/booth_bits
add wave sim:/BoothMultiplierTestbench/dut/control/rightMost
add wave sim:/BoothMultiplierTestbench/dut/control/differences

add wave sim:/BoothMultiplierTestbench/dut/load
add wave sim:/BoothMultiplierTestbench/dut/arth
add wave sim:/BoothMultiplierTestbench/dut/shiftRight

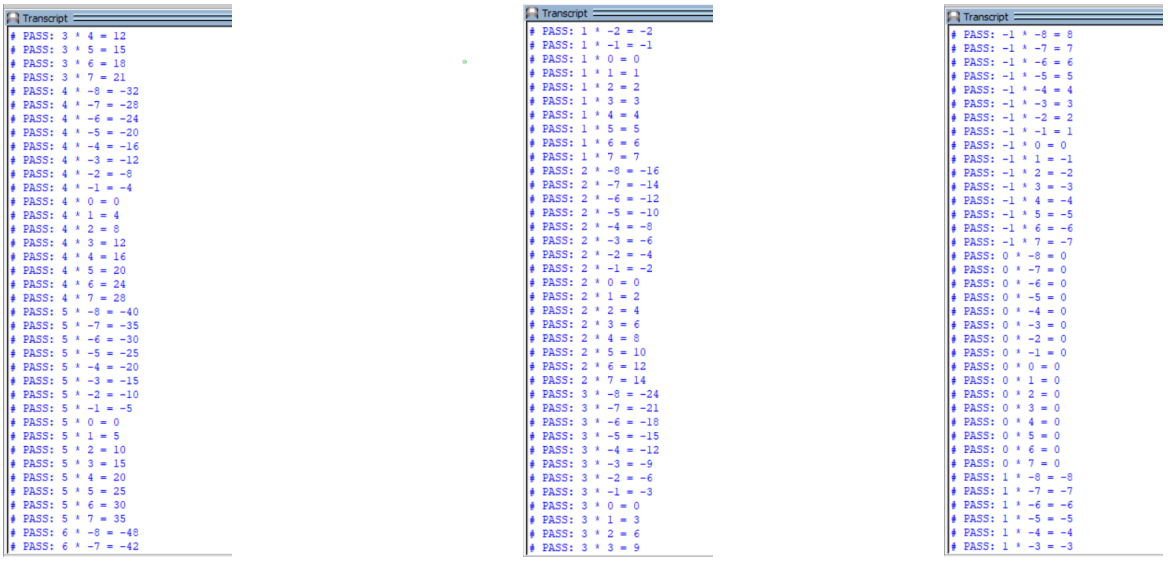
run -all
wave zoom full
```

کد ۵: فرمان‌های اجرای شبیه‌سازی و افزودن سیگنال‌ها به پنجره‌ی موج در ModelSim

متن رونوشت شبیه‌سازی برای هر ۲۵۶ حالت یک سطر PASS همراه با دو عملوند و حاصل‌ضرب چاپ می‌کند. تصاویر زیر این خروجی را از نخستین سطر، یعنی $(-8) \times (-8) = 64$، تا سطر پایانی دنبال می‌کنند.



شکل ۴: خروجی رونوشت برای مضروب‌های -8 تا -2



شکل ۵: خروجی رونوشت برای مضروب‌های -1 تا $+6$

گزارش پایانی نشان می‌دهد که هر ۲۵۶ آزمون اجرا شده و شمارندهٔ خطا صفر مانده است. پیام Break در سطر پایانی نیز مربوط به دستور $stop در انتهای میز آزمون است و خطا محسوب نمی‌شود.

```
# PASS: 6 * -6 = -36
# PASS: 6 * -5 = -30
# PASS: 6 * -4 = -24
# PASS: 6 * -3 = -18
# PASS: 6 * -2 = -12
# PASS: 6 * -1 = -6
# PASS: 6 * 0 = 0
# PASS: 6 * 1 = 6
# PASS: 6 * 2 = 12
# PASS: 6 * 3 = 18
# PASS: 6 * 4 = 24
# PASS: 6 * 5 = 30
# PASS: 6 * 6 = 36
# PASS: 6 * 7 = 42
# PASS: 7 * -8 = -56
# PASS: 7 * -7 = -49
# PASS: 7 * -6 = -42
# PASS: 7 * -5 = -35
# PASS: 7 * -4 = -28
# PASS: 7 * -3 = -21
# PASS: 7 * -2 = -14
# PASS: 7 * -1 = -7
# PASS: 7 * 0 = 0
# PASS: 7 * 1 = 7
# PASS: 7 * 2 = 14
# PASS: 7 * 3 = 21
# PASS: 7 * 4 = 28
# PASS: 7 * 5 = 35
# PASS: 7 * 6 = 42
# PASS: 7 * 7 = 49
# ----------------------------------------------------
# Tests  : 256
# Errors : 0
# ALL 256 SIGNED 4-BIT TESTS PASSED.
# ----------------------------------------------------
# Break in Module BoothMultiplierTestbench at C:/Users/ASUS/Downloads/dsd5/modelsim/BoothMultiplierTestbench.v line 90
```

شکل ۶: سطرهای پایانی رونوشت و گزارش قبولی هر ۲۵۶ حالت با صفر خطا

۷.۵ تحلیل شکل موج‌ها

الف) سیگنال‌های افزوده‌شده به پنجره‌ی موج

برای آنکه رفتار درونی مدار به‌طور کامل قابل ردیابی باشد، افزون بر واسط بیرونی، ثبات‌های داخلی مسیر داده و واحد کنترل نیز به پنجرهٔ موج افزوده شدند. جدول زیر معنای تک‌تک این سیگنال‌ها را گردآوری می‌کند.

سیگنال	معنا
clk	ساعت میز آزمون با دورهٔ ۱۰ نانوثانیه
str	پالس شروع؛ هم‌زمان اجازهٔ بارگذاری نیز هست
multiplicand	مضروب چهاربیتی علامت‌دار
multiplier	مضروب‌فیه چهاربیتی علامت‌دار
result	حاصل‌ضرب هشت‌بیتی، برابر الحاق $\{A[3:0], Q\}$
valid	اعلام آماده بودن حاصل‌ضرب
accumulator	انبارهٔ پنج‌بیتی A شامل بیت نگهبان
temp_multiplicand	مضروب M پس از بسط علامت به پنج بیت
q_reg	ثبات Q، مقدار جاری مضروب‌فیه
q_minus_one	تک‌بیت کمکی Q_{-1}
booth_register	الحاق ده‌بیتی علامت‌دار $\{A, Q, Q_{-1}\}$
state	حالت جاری ماشین کنترل
bits_remaining	تعداد بیت‌های پردازش‌نشده‌ی مضروب‌فیه
booth_bits	اندازه‌ی شیفت گام جاری
rightMost	طول دنباله‌ی بیت‌های همسان انتهای Q
differences	نقشه‌ی مرزهای تغییر میان بیت‌های مجاور Q
load	فرمان بارگذاری اوّلیّه‌ی ثبات‌ها
arth	فرمان اجرای مرحله‌ی حسابی
shiftRight	فرمان اجرای شیفت حسابی چندبیتی

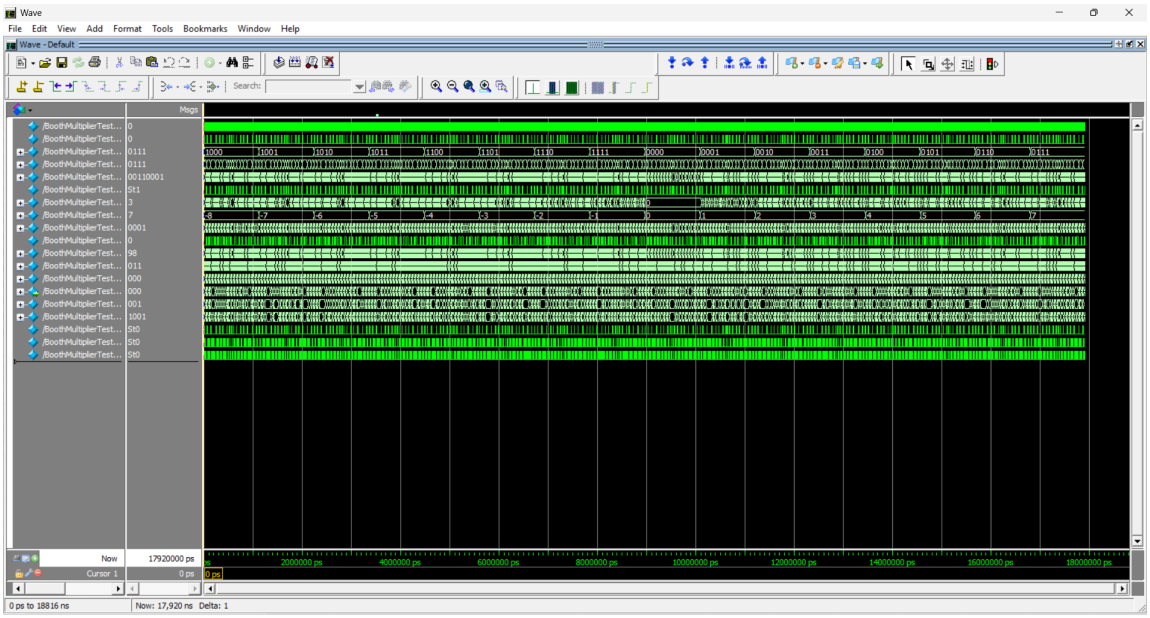
جدول ۱: سیگنال‌های افزوده‌شده به پنجره‌ی موج و معنای آن‌ها

یک نکتهٔ مهم در خواندن شکل موج‌ها آن است که result یک سیگنال ترکیبی و برابر الحاق چهار بیت کم‌ارزش انباره با ثبات Q است؛ بنابراین در طول اجرا و در میانهٔ تکرارها نیز پیوسته تغییر می‌کند و مقادیر میانی آن معنای حاصل‌ضرب ندارند. تنها مقداری از result معتبر است که هم‌زمان با یک بودن valid دیده شود.

ب) نمای کلّی شبیه‌سازی

شکل موج کامل، هر ۲۵۶ ضرب پیاپی را در بازهٔ صفر تا ۱۷,۹۲۰ نانوثانیه نشان می‌دهد. در ردیف multiplicand به‌روشنی دیده می‌شود که مضروب هر ۱۶ آزمون یک بار تغییر می‌کند و ترتیب $1000, 1001, \ldots, 1111, 0000, \ldots, 0111$ را می‌پیماید؛ یعنی همان پیمایش -8 تا $+7$ در متمّم دو. ردیف multiplier در همین بازه شانزده بار سریع‌تر تغییر می‌کند و به همین دلیل در این مقیاس فشرده به نظر می‌رسد. ردیف valid نیز یک قطار پالس منظّم است که هر پالس آن پایان یک ضرب را نشانه می‌گذارد.

از این نما یک نتیجهٔ کمّی به دست می‌آید: ۲۵۶ ضرب در ۱۷,۹۲۰ نانوثانیه، یعنی به‌طور میانگین ۷۰ نانوثانیه یا هفت پالس ساعت برای هر ضرب، شامل پالس شروع و پالس‌های میزبانی میز آزمون. عرض پالس‌های valid در سراسر نما یکسان نیست و این خود، شاهد بصری متغیّر بودن تعداد تکرارها بر حسب الگوی بیتی مضروب‌فیه است.

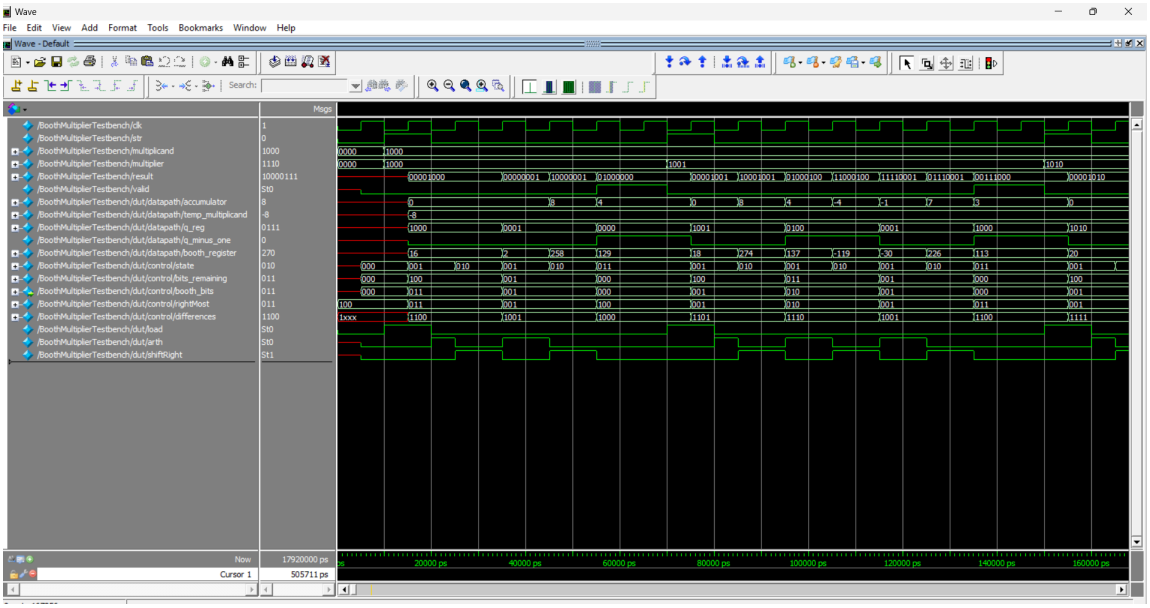


شکل ۷: شکل موج کامل شبیه‌سازی هر ۲۵۶ ضرب

ج) نخستین ضرب‌ها و حالت پیش از شروع

نمای بزرگ‌شدهٔ ابتدای شبیه‌سازی، بازهٔ صفر تا حدود ۱۷۰ نانوثانیه را پوشش می‌دهد. در سمت چپ این نما، پیش از نخستین پالس شروع، ثبات‌های درونی با رنگ قرمز و مقدار نامعیّن نمایش داده می‌شوند. علّت آن است که طرح ورودی بازنشانی ندارد و ثبات‌ها تا نخستین load مقداردهی نمی‌شوند. به همین دلیل differences در این بازه مقدار 1xxx دارد؛ زیرا ورودی multiplier واحد کنترل از q_reg نامعیّن مسیر داده تغذیه می‌شود و تنها بیت پرارزش آن، به‌سبب نقاب‌گذاری اجباری، معیّن است. مقدار rightMost نیز در همین بازه چهار است، چون حلقهٔ اولویّت هیچ بیت معیّن روشنی در جایگاه‌های پایین‌تر نمی‌یابد. این

رفتار مطلوب و بی‌ضرر است، زیرا نخستین لبهٔ str هر دو بخش مدار را به‌طور کامل مقداردهی می‌کند.



شکل ۸: نمای بزرگ‌شده‌ی نخستین ضرب‌ها؛ آغاز از حالت نامعیّن و اجرای $(-8) \times (-8)$ و $(-7) \times (-8)$

نخستین ضرب، حالت $(-8)\times(-8) = 64$ است که هر دو عملوند آن کمترین مقدار نمایش‌پذیر چهاربیتی‌اند و بحرانی‌ترین آزمون طرح به شمار می‌رود. جدول زیر، گام‌به‌گام مقادیر خوانده‌شده از شکل موج را دنبال می‌کند.

گام	حالت	A	Q	Q_{-1}	$\{A, Q, Q_{-1}\}$	booth_bits	bits_rem.
بارگذاری	ARITH	0	1000	0	16	3	4
حسابی	SHIFT	0	1000	0	16	3	4
شیفت 3	ARITH	0	0001	0	2	1	1
حسابی	SHIFT	8	0001	0	258	1	1
شیفت 1	DONE	4	0000	1	129	—	0

جدول ۲: دنبال‌کردن گام‌به‌گام ضرب $(-8) \times (-8) = 64$

تفسیر این جدول چنین است. پس از بارگذاری، $Q = 1000$ و $Q_{-1} = 0$ است؛ پس زوج $(0, 0)$ به دست می‌آید و مرحلهٔ حسابی هیچ تغییری در انباره نمی‌دهد. هم‌زمان واحد کنترل $\text{differences} = 1100$ و در نتیجه $\text{rightMost} = 3$ را محاسبه می‌کند، زیرا سه بیت انتهایی Q همگی صفرند. بنابراین سه گام از الگوریتم پایه در یک پالس ساعت فشرده می‌شود و $16 \ggg 3 = 2$ به دست می‌آید. اکنون $Q = 0001$ و $\text{bits_remaining} = 1$ است. در گام حسابی بعدی زوج $(1, 0)$ دیده می‌شود که به تفریق می‌انجامد: $A \leftarrow 0 - (-8) = 8$. این همان جایی است که بیت نگهبان کار خود را نشان می‌دهد، چون $+8$ در چهار بیت علامت‌دار جا نمی‌شد. سرانجام شیفت یک‌بیتی مقدار $258 \ggg 1 = 129$ را می‌سازد که به $A = 4$ و

$Q = 0000$ تجزیه می‌شود و چون $\text{bits_remaining} \leq \text{booth_bits}$ است، ماشین به حالت پایان می‌رود. حاصل نهایی $\text{result} = 01000000_2 = 64$ است که با انتظار تطابق دارد. نکتهٔ درخور توجّه آن است که این ضرب تنها با دو تکرار انجام شد، حال آنکه الگوریتم پایه چهار تکرار لازم داشت.

ضرب دوم، حالت $(-8) \times (-7) = 56$ است و چون الگوی بیتی 1001 دنبالهٔ بلند همسان ندارد، تکرارهای بیشتری لازم می‌شود.

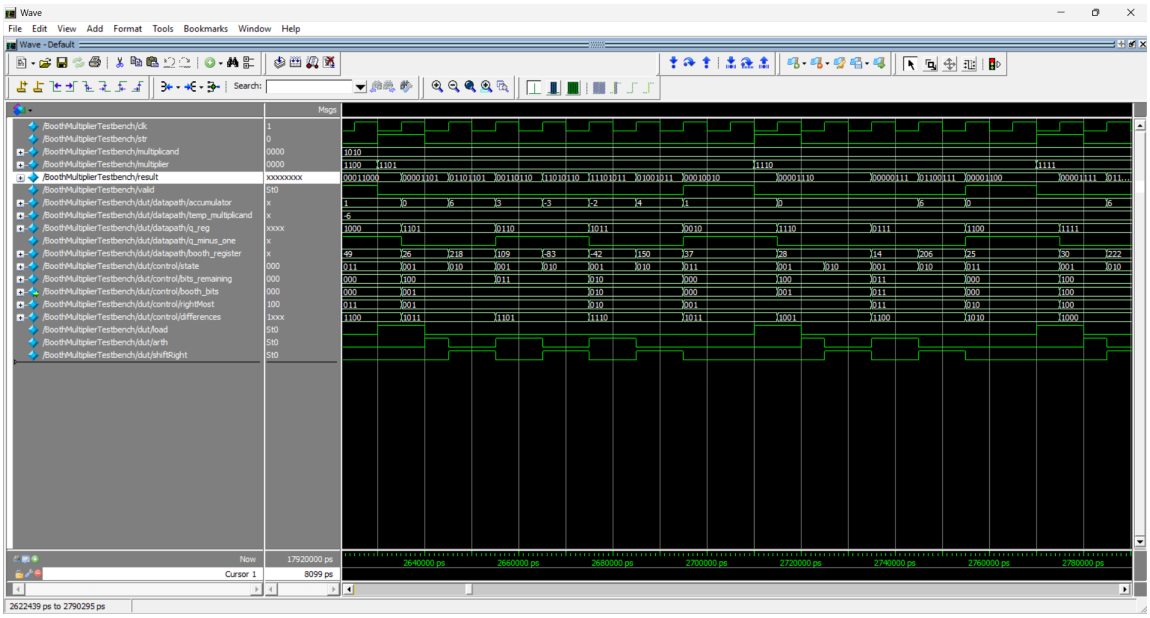
گام	حالت	A	Q	Q_{-1}	$\{A, Q, Q_{-1}\}$	booth_bits	bits_rem.
بارگذاری	ARITH	0	1001	0	18	1	4
حسابی	SHIFT	8	1001	0	274	1	4
شیفت 1	ARITH	4	0100	1	137	2	3
حسابی	SHIFT	-4	0100	1	-119	2	3
شیفت 2	ARITH	-1	0001	0	-30	1	1
حسابی	SHIFT	7	0001	0	226	1	1
شیفت 1	DONE	3	1000	1	113	—	0

جدول ۳: دنبال‌کردن گام‌به‌گام ضرب $(-8) \times (-7) = 56$

در این دنباله هر سه رفتار مرحلهٔ حسابی دیده می‌شود. در گام نخست زوج $(1, 0)$ به تفریق می‌انجامد و A از صفر به 8 می‌رسد. در گام دوم زوج $(0, 1)$ دیده می‌شود که جمع است و A از 4 به -4 می‌رود. در گام سوم دوباره تفریق رخ می‌دهد و A از -1 به 7 می‌رسد. اندازهٔ شیفت نیز در این سه گام به‌ترتیب 1، 2 و 1 است؛ شیفت دوبیتی از آنجا ناشی می‌شود که پس از نخستین شیفت، $Q = 0100$ شده و دو بیت انتهایی آن همسان‌اند. شیفت حسابی روی مقادیر منفی نیز به‌درستی کار می‌کند: $-119 \ggg 2 = -30$ و بیت علامت در جایگاه‌های پرارزش تکرار شده است. حاصل نهایی $\text{result} = 00111000_2 = 56$ است.

د) بهترین حالت زمانی: مضروب‌فیه با بیت‌های یکسان

نمای بزرگ‌شدهٔ دوم، بازهٔ ۲٫۶۲۲ تا ۲٫۷۹۰ نانوثانیه را با مضروب ثابت $1010_2 = -6$ نشان می‌دهد. در سمت راست این نما، مضروب‌فیه برابر $1111_2 = -1$ است و بهترین حالت زمانی طرح رخ می‌دهد. چون هر چهار بیت Q یکسان‌اند، $\text{differences} = 1000$ و در نتیجه $\text{rightMost} = 4$ و $\text{booth_bits} = 4$ می‌شود؛ یعنی هر چهار بیت در یک پالس ساعت مصرف می‌شوند.



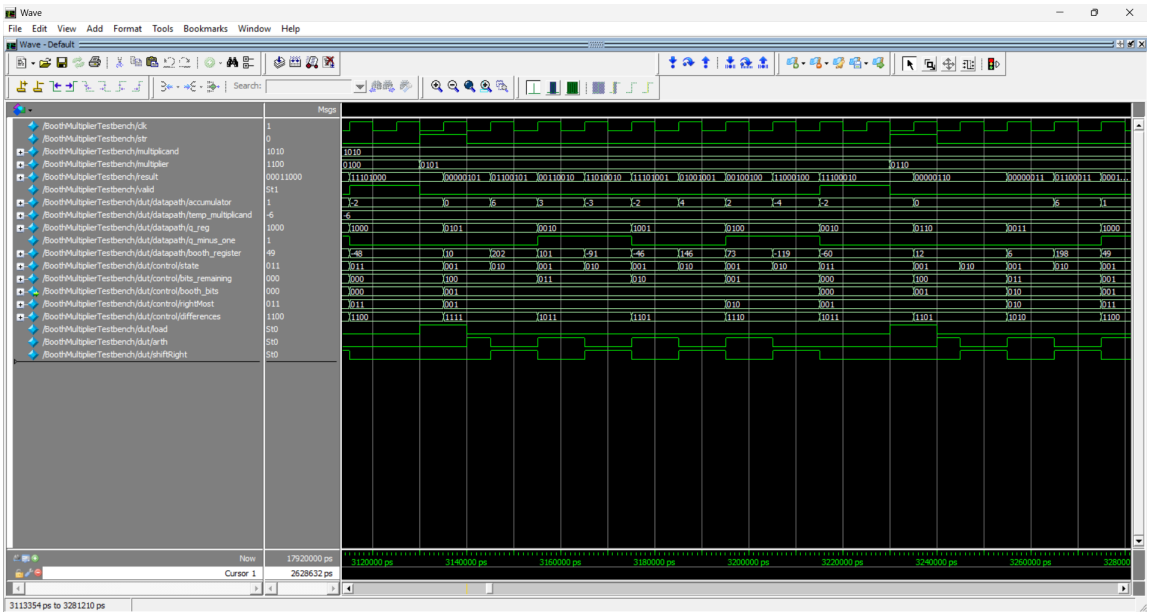
شکل ۹: نمای بزرگ‌شده‌ی بازه‌ی میانی و بهترین حالت زمانی با مضروب‌فیه 1111

روند کار چنین است: پس از بارگذاری، زوج $(1,0)$ به تفریق می‌انجامد و $A \leftarrow 0-(-6)=6$ می‌شود؛ در همین لحظه $\{A,Q,Q_{-1}\}=222$ در شکل موج دیده می‌شود. سپس یک شیفت چهاربیتی $222 \ggg 4=13$ را می‌سازد که به $A=0$ و $Q=0110$ تجزیه می‌شود و چون $\text{bits_remaining} \leq \text{booth_bits}$ است، ماشین بی‌درنگ به حالت پایان می‌رود. حاصل $\text{result}=00000110_2=6$ است که همان $(-1)\times(-6)$ است. این ضرب تنها با یک تکرار و در دو پالس ساعت انجام شد، حال آنکه الگوریتم پایه چهار تکرار و هشت پالس لازم داشت؛ یعنی تسریعی چهاربرابر.

در همین نما، ضرب پیش از آن با مضروب‌فیه $1110_2=-2$ را نیز می‌توان دنبال کرد که در حالت پایان مقدار $\text{result}=00001100_2=12$ را نشان می‌دهد و با $(-2)\times(-6)=12$ سازگار است.

ه) بدترین حالت زمانی: مضروب‌فیه با بیت‌های متناوب

نمای بزرگ‌شدهٔ سوم، بازهٔ ۳,۱۱۳ تا ۳,۲۸۱ نانوثانیه را با همان مضروب -6 نشان می‌دهد. در این بازه مضروب‌فیه برابر $0101_2=+5$ است؛ الگویی که بیت‌های آن یکی در میان تغییر می‌کنند و بنابراین بدترین حالت زمانی طرح به شمار می‌رود.



شکل ۱۰: نمای بزرگ‌شده‌ی بدترین حالت زمانی با مضروب‌فیه 0101

گام	حالت	A	Q	Q_{-1}	$\{A, Q, Q_{-1}\}$	booth_bits	bits_rem.
بارگذاری	ARITH	0	0101	0	10	1	4
حسابی	SHIFT	6	0101	0	202	1	4
شیفت 1	ARITH	3	0010	1	101	1	3
حسابی	SHIFT	-3	0010	1	-91	1	3
شیفت 1	ARITH	-2	1001	0	-46	1	2
حسابی	SHIFT	4	1001	0	146	1	2
شیفت 1	ARITH	2	0100	1	73	1	1
حسابی	SHIFT	-4	0100	1	-119	1	1
شیفت 1	DONE	-2	0010	0	-60	—	0

جدول ۴: دنبال‌کردن گام‌به‌گام ضرب $(-6) \times (+5) = -30$

در سراسر این دنباله، differences پیاپی مقادیر 1111، 1011، 1101 و 1110 می‌گیرد و در هر چهار گام کم‌ارزش‌ترین بیت روشن آن در جایگاه نخست یا نزدیک به آن قرار دارد؛ ازاین‌رو booth_bits همواره یک می‌ماند و مدار دقیقاً مانند الگوریتم پایه چهار تکرار انجام می‌دهد. مقدار نهایی $\text{result} = 11100010_2$ است که در متمّم دو برابر -30 و با $(-6) \times (+5)$ سازگار است. این نما نشان می‌دهد که مدار در بدترین حالت نیز درست کار می‌کند و تنها مزیّت زمانی خود را از دست می‌دهد.

و) جمع‌بندی رفتار مشاهده‌شده

مجموع چهار شکل موج بررسی‌شده، همهٔ حالت‌های ممکن مدار را پوشش می‌دهد:

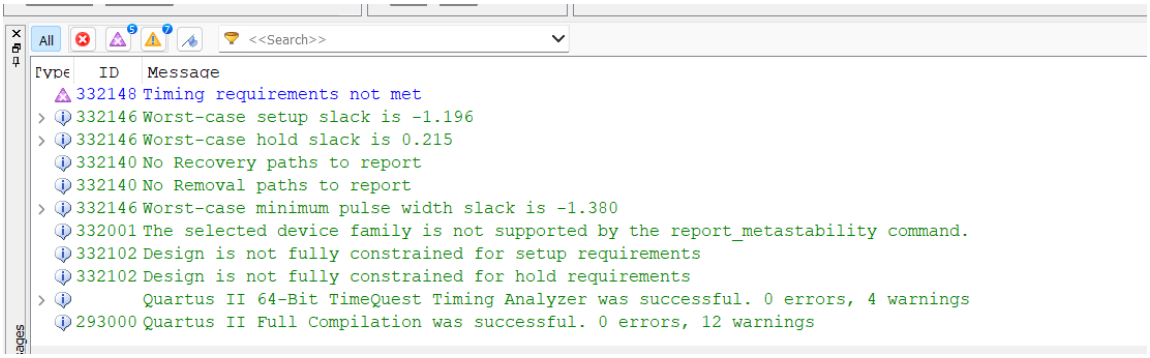
- هر چهار حالت ماشین کنترل، یعنی IDLE پیش از نخستین شروع، ARITH، SHIFT و DONE، در نماها دیده و دنبال شدند.
- هر سه رفتار ممکن مرحلهٔ حسابی، یعنی جمع برای زوج $(0,1)$، تفریق برای زوج $(1,0)$ و حفظ مقدار برای زوج‌های $(0,0)$ و $(1,1)$، مشاهده شد.
- اندازه‌های شیفت 1، 2 و 4 در نماهای گوناگون رخ داد و شیفت حسابی هم روی مقادیر مثبت و هم روی مقادیر منفی انباره درست عمل کرد.
- هر دو حدّ زمانی طرح، یعنی یک تکرار برای مضروب‌فیه یکنواخت و چهار تکرار برای مضروب‌فیه متناوب، دیده شد.
- همبستگی load، arth، shiftRight و valid با حالت‌های ماشین در همهٔ نماها برقرار بود و هیچ‌گاه دو فرمان هم‌زمان فعّال نشدند.

۸.۵ سنتز و پیاده‌سازی نرم‌افزاری

سه پروندهٔ سنتزپذیر طرح، بدون میز آزمون، در Quartus II 13.0.1 و برای قطعهٔ EP2C20F484C7 از خانوادهٔ Cyclone II همگردانی و سنتز شدند. خلاصهٔ منابع مصرفی چنین است: ۸۸ عنصر منطقی شامل ۸۴ تابع ترکیبی و ۲۰ ثبات اختصاصی، ۱۹ پایه و هیچ بیت حافظه یا حلقهٔ قفل‌شده در فاز. مصرف هر سه دستهٔ منابع کمتر از یک درصد ظرفیّت قطعه است.

دو نکته در این گزارش شایان توجّه است. نخست آنکه شمار ضرب‌کننده‌های نه‌بیتی توکار[۲۰] مصرف‌شده صفر است؛ یعنی سنتزکننده هیچ بلوک ضرب آمادهٔ تراشه را به کار نگرفته و کلّ عمل ضرب واقعاً با همان مسیر داده و واحد کنترل طرّاحی‌شده انجام می‌شود. دوم آنکه شمار ثبات‌ها با انتظار سازگار است: در مسیر داده پنج بیت انباره، پنج بیت مضروب، چهار بیت Q و یک بیت Q_{-1} و در واحد کنترل سه بیت bits_remaining در مجموع هجده بیت می‌شوند و دو بیت باقی‌مانده به ثبات حالت تعلّق دارد. ثبات حالت گرچه سه‌بیتی تعریف شده، تنها چهار مقدار می‌گیرد؛ ازاین‌رو سنتزکننده به‌درستی بیت پرارزش آن را که همواره صفر است حذف کرده و ۲۰ ثبات گزارش شده است.

[20] Embedded Multiplier



شکل ۱۱: پیام‌های پایانی همگردانی موفّق و هشدارهای تحلیل زمانی

همگردانی کامل با صفر خطا و ۱۲ هشدار به پایان رسید. هشدار بحرانی Timing requirements not met در تصویر بالا نیازمند توضیح است. برای این پروژه پروندهٔ قیود طراحی[۲۱] تعریف نشده است؛ در چنین حالتی، تحلیل‌گر زمانی[۲۲] به‌طور خودکار قید create_clock -period 1.000 -name clk clk را استنتاج می‌کند، یعنی ساعتی با دورهٔ یک نانوثانیه معادل یک گیگاهرتز. طبیعی است که طرح در برابر چنین قید تحمیلی و غیرواقع‌بینانه‌ای حاشیهٔ زمانی[۲۳] منفی نشان دهد؛ بدترین حاشیهٔ برپایی در مدل تند برابر -1.196 نانوثانیه، یعنی همان مقداری که در تصویر بالا دیده می‌شود، و در مدل کند برابر -5.052 نانوثانیه گزارش شده است. هشدار کمینهٔ پهنای پالس نیز از همین ریشه می‌آید.

معیار درست ارزیابی در این وضعیّت، بیشینه‌بسامد[۲۴] گزارش‌شده است. تحلیل‌گر زمانی برای مدل کند مقدار $165.23\,\text{MHz}$ را گزارش می‌کند و همین عدد به‌عنوان بیشینه‌بسامد مقیّد نیز آمده است. ساعت ۱۰۰ مگاهرتزی که در میز آزمون به کار رفته، پایین‌تر از این حدّ است و بنابراین شبیه‌سازی انجام‌شده با توان واقعی طرح سازگار است. حاشیهٔ نگهداری در هر دو مدل کند و تند مثبت است و بنابراین مسیرهای کوتاه مشکلی ندارند.

با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۹.۵ نتایج

- ضرب‌کنندهٔ بوث برای عملوندهای چهاربیتی علامت‌دار پیاده‌سازی و مطابق دستورکار، مسیر داده و واحد کنترل در دو پیمانهٔ جداگانه طراحی و سپس به یکدیگر متّصل شدند.

[21] Synopsys Design Constraints
[22] TimeQuest Timing Analyzer
[23] Slack
[24] Maximum Frequency (Fmax)

- شیفت‌دهنده به‌گونه‌ای طراحی شد که در هر پالس ساعت تا چهار بیت جابه‌جایی حسابی انجام دهد و اندازهٔ شیفت به‌صورت پویا از روی طول دنبالهٔ بیت‌های همسان مضروب‌فیه محاسبه می‌شود.
- آزمون فراگیر هر ۲۵۶ زوج ورودی ممکن بدون هیچ خطا و بدون هیچ مورد وقفهٔ زمانی به پایان رسید.
- حالت مرزی $8-$ که در آن تفریق بوث به مقدار غیرقابل نمایش $8+$ می‌انجامد، با افزودن یک بیت نگهبان به انباره و ثبات مضروب به‌درستی رسیدگی شد.
- تعداد تکرارها بسته به الگوی بیتی مضروب‌فیه میان یک تا چهار تغییر می‌کند؛ در بهترین حالت مشاهده‌شده تسریع چهاربرابر نسبت به الگوریتم پایه به دست آمد.
- سنتز با ۸۸ عنصر منطقی، ۲۰ ثبات و ۱۹ پایه، همگی کمتر از یک درصد ظرفیّت قطعه، و بدون استفاده از ضرب‌کنندهٔ توکار تراشه انجام شد.
- بیشینه‌بسامد گزارش‌شده $165.23\,\mathrm{MHz}$ است و ساعت ۱۰۰ مگاهرتزی شبیه‌سازی حاشیهٔ کافی دارد.

نتیجه‌گیری

در این آزمایش یک ضرب‌کنندهٔ علامت‌دار بر پایهٔ الگوریتم بوث طراحی و ارزیابی شد. جداسازی مسیر داده از واحد کنترل، هم خواستهٔ دستورکار بود و هم آزمودن و خواندن طرح را ساده کرد؛ چنان‌که در شکل موج‌ها می‌توان تصمیم‌های واحد کنترل را مستقل از تغییرات ثبات‌های مسیر داده دنبال کرد. تعمیم شیفت یک‌بیتی به شیفت چندبیتی، با تکیه بر این مشاهده که دنباله‌های بیت همسان در مضروب‌فیه هیچ عمل حسابی لازم ندارند، تعداد تکرارها را از مقدار ثابت چهار به مقداری متغیّر میان یک تا چهار کاهش داد و همان تسریعی را فراهم کرد که دستورکار خواسته بود. آزمون فراگیر ۲۵۶ حالته، بررسی گام‌به‌گام شکل موج‌ها در بهترین و بدترین حالت زمانی و گزارش‌های سنتز و تحلیل زمانی، در مجموع درستی عملکرد و پیاده‌سازی‌پذیری طرح را تأیید می‌کنند.